

GROUP 15

TOMBSTONE SERVICES



TABLE OF CONTENTS

TABLE OF CONTENTS	2
APPENDIX A: GROUP PROJECT COVER SHEET	4
APPENDIX B: GROUP PLAGIARISM STATEMENT.....	4
Group Plagiarism Statement.....	4
INTRODUCTION.....	6
PART 1: RELATIONAL DATABASE DESIGN	6
<i>1.1 Entity-Relationship Diagram (ERD)</i>	<i>6</i>
<i>1.2 Normalisation to First Normal Form (1NF).....</i>	<i>6</i>
Unnormalised Order Form (UNF)	Error! Bookmark not defined.
Result — 1NF	Error! Bookmark not defined.
<i>1.3 Normalisation to Second Normal Form (2NF).....</i>	<i>Error! Bookmark not defined.</i>
Result — 2NF	Error! Bookmark not defined.
<i>1.4 Relational Schema (Normalised to Third Normal Form (3NF))</i>	<i>7</i>
<i>1.5 Sample Data</i>	<i>7</i>
CUSTOMER	7
CEMETERY	8
SUPPLIER.....	8
GRANITE_TYPE	8
EMPLOYEE.....	8
PRODUCT.....	8
ORDER.....	8
INVOICE.....	9
DELIVERY	9
ORDER_ITEM	9
DECEASED.....	9
MANUFACTURING	9
INVENTORY	10
MATERIAL.....	10
SLAB_INVENTORY.....	10
SUPPLIER_MATERIAL.....	10
PRODUCT_MATERIAL.....	10
<i>1.6 SQL Scripts.....</i>	<i>10</i>
(a) Table creation (DDL) — in dependency order	11
(b) Sample data population (DML)	13
PART 2: DATA WAREHOUSE DESIGN	13
<i>2.1 Facts and Dimension Tables</i>	<i>14</i>
Fact Table	14

Dim_Date	14
Dim_Product	14
Dim_Order.....	15
Dim_Customer	15
<i>2.2 Design Choice Explained.....</i>	15
PART 3: DATA MARTS.....	15
<i>3.1 Data Mart 1: Sales Performance Mart (Core Process)</i>	15
Key indicators / metrics.....	15
Derivation SQL (from the Part 2 warehouse)	15
Sample output (July 2026).....	15
<i>3.2 Data Mart 2: Order Fulfilment Mart (Secondary/Management Process)</i>	16
Key indicators / metrics.....	16
Derivation SQL (from the Part 2 warehouse)	16
Sample output (July 2026).....	16
<i>Summary of Business Value</i>	16
REFERENCES.....	17



APPENDIX A: GROUP PROJECT COVER SHEET

Group No.: 15

Group Name: TOMBSTONE SERVICES

Project Name: BISB301

Please complete this form and submit it as a cover sheet with your project submission. One submission is required per group. The details completed below must be correct.

Student No.	Surname & Initials	Task Completed	Signature
22416369	AMAHLE SIBISI	ERD	AS
22124072	BANELE DUBE	NORMALIZATION	BD
22239183	MHLAMBI ZO	SAMPLE DATA & SQL	ZOM
22268369	CELE NOZUZO	DATA WAREHOUSE	NC
22310593	NDLELA S	DATA MART	SN
22206931	NZAMA Z	REPORT AND DOCUMENT	ZN
22453696	NHLANGOTHI PP	REPORT AND DOCUMENT	PPN
22064268	SOKHULU KM	REPORT AND DOCUMENT	KMS
22410105	ZONDI S	DATA MART	ZS

APPENDIX B: GROUP PLAGIARISM STATEMENT

FACULTY OF ACCOUNTING AND INFORMATICS – DEPARTMENT OF INFORMATION TECHNOLOGY

Group Plagiarism Statement

You are guilty of plagiarism if you copy something from a book, article, or website without acknowledging the source and passing it off as your own. In effect, you are stealing something that belongs to someone else. This is not only the case when you copy work word-by-word (verbatim), but also when you submit someone else's work in a slightly altered form (paraphrase) or use a line of argument without acknowledging it. You are not allowed to use another student's past written work. You are also not allowed to let anybody copy your work to pass it off as his/her work.

Students who commit plagiarism will get 0 (zero) for the plagiarised work, without the opportunity to resubmit, AND the matter may also be referred to the Dean for disciplinary action. Plagiarism is regarded as a serious contravention of the rules and can lead to expulsion from this and other universities.

This declaration must be completed and submitted to your respective group lecturer for all phases of the project.

No.	Student No.	Student Initials & Surname	Signature
1	22239183	ZO Mhlambi	ZM
2	22268369	Nozuzo Cele	NC
3	22416369	Amahle Sibisi	AS
4	22124072	Banele Dube	BD

No.	Student No.	Student Initials & Surname	Signature
5	22206931	Zazi Nzama	ZN
6	22310593	Sandiso Ndlela	SN
7	22064268	Kwanele Sokhulu	KS
8	22453696	Nhlangothi Phindi	NP
9	22410105	Zondi S	ZS

- We understand what plagiarism is and we are aware of the DUT's policy in this regard.
- We declare that this tutorial/project is our own work.
- Where other people's work has been used (either from a printed source, the internet, or any other source) this has been carefully acknowledged and referenced following departmental requirements.
- We have not used other students' past work to hand in as our own.
- We have not allowed and will not allow anyone to copy our work to pass it off as their work.

Because the business is juggling customers, orders, suppliers, materials and production at the same time, it needs a proper system to keep track of all of this and simple reports that let management see how sales and production are going. That is what this project sets out to build: a relational database (Part 1), a data warehouse (Part 2), and two data marts (Part 3).

Part 1 covers the day-to-day operational database for the company, a business that manufactures and installs granite tombstones, headstones and memorial monuments for customers arranging a memorial for a deceased family member. It includes the Entity-Relationship Diagram, the normalisation process from an unnormalised design through First, Second and Third Normal Form, sample data (3-5 rows per table) and the SQL scripts used to create and populate the tables.

Legend: **PK** = Primary Key **FK** = Foreign Key Arrow points from the "1" (parent) side to the "M" / "1" (child) side of each relationship.
Cardinality labels (1 / M) are shown at each end of the connecting line - e.g. CUSTOMER (1) → ORDER (M) means one customer can place many orders.

1.2 Normalisation to First Normal Form (1NF)

- ORDER_FORM (OrderID PK, OrderDate, OrderStatus, CustomerName, CustomerPhone, CustomerEmail, CustomerAddress)
- ORDER_ITEM (OrderID PK/FK, ProductID PK/FK, ProductName, ProductType, Height_cm, Width_cm, Quantity, UnitPrice)

1.3 Relational Schema (Normalised to Second Normal Form (2NF))

- PRODUCT (ProductID PK, ProductName, ProductType, Height_cm, Width_cm, UnitPrice)
- ORDER_ITEM (OrderItemID PK, Quantity, UnitPrice, OrderID FK, ProductID FK)
- ORDER (OrderID PK, OrderDate, OrderStatus, CustomerName, CustomerPhone, CustomerEmail, CustomerAddress)

1.4 Relational Schema (Normalised to Third Normal Form (3NF))

- CUSTOMER (CustomerID PK, FirstName, LastName, PhoneNumber, Email, Address)
- ORDER (OrderID PK, OrderDate, OrderStatus, CustomerID FK)
- INVOICE (InvoiceID PK, InvoiceDate, TotalAmount, AmountPaid, BalanceDue, InvoiceStatus, OrderID FK)
- DELIVERY (DeliveryID PK, DeliveryDate, DeliveryAddress, DriverName, DeliveryStatus, OrderID FK)
- SUPPLIER (SupplierID PK, SupplierName, PhoneNumber, Email, Address)
- PRODUCT (ProductID PK, ProductName, ProductType, Height_cm, Width_cm, UnitPrice)
- ORDER_ITEM (OrderItemID PK, Quantity, UnitPrice, OrderID FK, ProductID FK)
- DECEASED (DeceasedID PK, DeceasedName, DateOfBirth, DateOfDeath, CustomerID FK, CemeteryID FK)
- CEMETERY (CemeteryID PK, CemeteryName, Location, ContactNumber)
- MANUFACTURING (ManufacturingID PK, ManufacturingDate, CompletionDate, ManufacturingStatus, QuantityProduced, ManufacturingNotes, OrderID FK, ProductID FK, EmployeeID FK)
- EMPLOYEE (EmployeeID PK, EmployeeName, JobTitle, PhoneNumber, Email)
- INVENTORY (InventoryID PK, QuantityInStock, StockStatus, LastUpdated, ProductID FK)
- MATERIAL (MaterialID PK, MaterialName, UnitCost, SupplierID FK)
- GRANITE_TYPE (GraniteTypeID PK, GraniteName, Colour, Origin)
- SLAB_INVENTORY (SlabInventoryID PK, SlabCode, Size, Quantity, UnitCost, GraniteTypeID FK)
- SUPPLIER_MATERIAL (SupplierMaterialID PK, SupplierID FK, MaterialID FK)
- PRODUCT_MATERIAL (ProductMaterialID PK, ProductID FK, MaterialID FK)

1.5 Sample Data

CUSTOMER

ID	First Name	Last Name	Phone	Email	Address
C001	Zanele	Zungu	0627885467	zunguz@gmail.com	Durban



ID	First Name	Last Name	Phone	Email	Address
C002	Jomo	Zwane	0784456790	jzwane@gmail.com	Pinetown
C003	Emily	Armish	0834521345	smith2@gmail.com	Ballito

CEMETERY

ID	Name	Location	Contact
CEM1	Stellawood Cemetery	Umbilo, Durban	0312758908
CEM2	Pinetown Cemetery	Fields Hill, Pinetown	0312789046
CEM3	Umhlali Cemetery	Salt Rock Road, Ballito	0324783094

SUPPLIER

ID	Name	Number	Email	Address
SUP1001	David Blom	0678453210	blomd@gmail.com	Cape Town
SUP1002	Zakes Nkosi	0897634569	zakesnkosi@gmail.com	Durban
SUP1003	Albert Yaman	0725789656	yaman@gmail.com	Johannesburg

GRANITE_TYPE

ID	Name	Colour	Origin
GR01	Nero Rustenburg	Dark Grey	South Africa
GR02	Zimbabwe Black	Deep Intense Black	South Africa
GR03	African Red	Deep Red	South Africa

EMPLOYEE

ID	Name	Job	Phone	Email
E101	Zinhle Buthelezi	Sales Consultant	0867845637	zbuthelezi@gmail.com
E102	Kevin Moodley	Master Stone Carver	0723678907	moodleyk@gmail.com
E103	Lungani Cele	Installation Specialist	0824789084	cele24@gmail.com

PRODUCT

ID	Product Name	Product Type	Height cm	Width cm	Unit Price
P01	Rustenburg Grey Flat Marker	Lawn Plaque	10	60	4500.00
P02	Zimbabwe Black Classic	Headstone	80	65	12000.00
P03	African Red Double Memorial	Full Monument	110	150	3500.00

ORDER

ID	Order Date	Order Status	CustomerID
1001	2026-07-06	Quality Check	C001
1002	2026-07-08	In Production	C002
1003	2026-07-13	Ready	C003

INVOICE

ID	Invoice Date	Total Amount	Amount Paid	Balance Due	Status	OrderID
N1	2026-07-06	4500	2000	2500	Unpaid	1001
N2	2026-07-08	12000	6000	6000	Partially Paid	1002
N3	2026-07-13	3500	3500	0	Paid	1003

DELIVERY

ID	Delivery Date	Address	Driver	Status	OrderID
DEL01	2026-07-16	Umbilo, Durban	Lungisani Cele	Delivered	1001
DEL02	2026-08-03	Fields Hill, Pinetown	Lungisani Cele	Delivered	1002
DEL03	NULL	Salt Rock, Ballito	Lungisani Cele	Scheduled	1003

ORDER_ITEM

ID	Quantity	Unit Price	OrderID	ProductID
1	1	4500	1001	P01
2	1	12000	1002	P02
3	1	3500	1003	P03

DECEASED

ID	Name	DOB	DOD	CustomerID	CemeteryID
D01	Siphila Zungu	1995-02-28	2026-06-25	C001	CEM1
D02	Zwelihle Zwane	1942-08-08	2026-07-01	C002	CEM2
D03	World Armish	2002-04-17	2026-07-10	C003	CEM3

MANUFACTURING

ID	Man. Date	Completion	Status	Qty	Notes	OrderID	ProductID
MFG001	2026-07-09	2026-07-14	Completed	1	Moved to dispatch bay	1001	P01
MFG002	2026-06-12	2026-07-28	Completed	1	Etching completed	1002	P02

ID	Man. Date	Completion	Status	Qty	Notes	OrderID	ProductID
MFG003	2026-07-15	NULL	In Progress	1	Raw stone slab loaded	1003	P02

INVENTORY

ID	Qty In Stock	Stock Status	Last Updated	ProductID
1	4	In Stock	2026-07-14	P01
2	2	Low Stock	2026-07-28	P02
3	6	In Stock	2026-07-13	P03

MATERIAL

ID	Name	Unit Cost	SupplierID
M1	Industrial Sandblast Diamond 25kg	1450	SUP1001
M2	Monumental Gold Leaf Foil 23 Karat	2800	SUP1002
M3	Weatherproof EpoxyStone Sealant 5L	850	SUP1003

SLAB_INVENTORY

ID	Code	Size	Quantity	Unit Cost	GraniteTypeID
Slab01	Rst1	250x140x3cm	15	4200	GR01
Slab02	Zim2	240x120x5cm	8	7800	GR02
Slab03	Afr3	260x150x5cm	5	6100	GR03

SUPPLIER_MATERIAL

ID	SupplierID	MaterialID
SM01	SUP1001	M1
SM02	SUP1002	M2
SM03	SUP1003	M3

PRODUCT_MATERIAL

ID	ProductID	MaterialID
PM001	P01	M1
PM002	P02	M2
PM003	P03	M3

1.6 SQL Scripts

The full script is provided as a separate, executable file named SQLqueries-tables.sql, submitted alongside this report. It contains every CREATE TABLE and INSERT INTO statement, organised in dependency order (parent tables are created before the child tables that reference them with a FOREIGN

KEY) and grouped table-by-table below for readability. The script was tested end-to-end and executes without errors.

(a) Table creation (DDL) — in dependency order

```
-- 1.1 CUSTOMER (no dependencies)
CREATE TABLE Customer (
  CustomerID VARCHAR(10) PRIMARY KEY NOT NULL,
  FirstName  VARCHAR(50),
  LastName   VARCHAR(50),
  PhoneNumber VARCHAR(20),
  Email      VARCHAR(100),
  Address    VARCHAR(150)
);

-- 1.2 CEMETERY (no dependencies)
CREATE TABLE Cemetery (
  CemeteryID VARCHAR(10) PRIMARY KEY NOT NULL,
  CemeteryName VARCHAR(100),
  Location     VARCHAR(150),
  ContactNumber VARCHAR(20)
);

-- 1.3 SUPPLIER (no dependencies)
CREATE TABLE Supplier (
  SupplierID VARCHAR(10) PRIMARY KEY NOT NULL,
  SupplierName VARCHAR(100),
  PhoneNumber VARCHAR(20),
  Email        VARCHAR(100),
  Address      VARCHAR(150)
);

-- 1.4 GRANITE TYPE (no dependencies)
CREATE TABLE Granite_Type (
  GraniteTypeID VARCHAR(10) PRIMARY KEY NOT NULL,
  GraniteName   VARCHAR(100),
  Colour        VARCHAR(50),
  Origin        VARCHAR(100)
);

-- 1.5 EMPLOYEE (no dependencies)
CREATE TABLE Employee (
  EmployeeID VARCHAR(10) PRIMARY KEY NOT NULL,
  EmployeeName VARCHAR(50),
  JobTitle    VARCHAR(50),
  PhoneNumber VARCHAR(20),
  Email       VARCHAR(100)
);

-- 1.6 PRODUCT (no dependencies)
CREATE TABLE Product (
  ProductID VARCHAR(10) PRIMARY KEY NOT NULL,
  ProductName VARCHAR(100),
  ProductType VARCHAR(100),
  Height_cm   INT,
  Width_cm    INT,
  UnitPrice   DECIMAL(10,2)
);

-- 1.7 ORDER (depends on Customer)
CREATE TABLE `Order` (
  OrderID INT PRIMARY KEY NOT NULL,
```

```
OrderDate DATE,
OrderStatus VARCHAR(30),
CustomerID VARCHAR(10),
FOREIGN KEY (CustomerID) REFERENCES Customer (CustomerID)
);
```

-- 1.8 INVOICE (depends on Order)

```
CREATE TABLE Invoice (
InvoiceID VARCHAR(10) PRIMARY KEY NOT NULL,
InvoiceDate DATE,
TotalAmount DECIMAL(10,2),
AmountPaid DECIMAL(10,2),
BalanceDue DECIMAL(10,2),
InvoiceStatus VARCHAR(30),
OrderID INT,
FOREIGN KEY (OrderID) REFERENCES `Order` (OrderID)
);
```

-- 1.9 DELIVERY (depends on Order)

```
CREATE TABLE Delivery (
DeliveryID VARCHAR(10) PRIMARY KEY NOT NULL,
DeliveryDate DATE,
DeliveryAddress VARCHAR(150),
DriverName VARCHAR(100),
DeliveryStatus VARCHAR(30),
OrderID INT,
FOREIGN KEY (OrderID) REFERENCES `Order` (OrderID)
);
```

-- 1.10 ORDER_ITEM (depends on Order, Product)

```
CREATE TABLE Order_Item (
OrderItemID INT PRIMARY KEY NOT NULL,
Quantity INT,
UnitPrice DECIMAL(10,2),
OrderID INT,
ProductID VARCHAR(10),
FOREIGN KEY (OrderID) REFERENCES `Order` (OrderID),
FOREIGN KEY (ProductID) REFERENCES Product (ProductID)
);
```

-- 1.11 DECEASED (depends on Customer, Cemetery)

```
CREATE TABLE Deceased (
DeceasedID VARCHAR(10) PRIMARY KEY NOT NULL,
DeceasedName VARCHAR(50),
DateOfBirth DATE,
DateOfDeath DATE,
CustomerID VARCHAR(10),
CemeteryID VARCHAR(10),
FOREIGN KEY (CustomerID) REFERENCES Customer (CustomerID),
FOREIGN KEY (CemeteryID) REFERENCES Cemetery (CemeteryID)
);
```

-- 1.12 MANUFACTURING (depends on Order, Product, Employee)

```
CREATE TABLE Manufacturing (
ManufacturingID VARCHAR(10) PRIMARY KEY NOT NULL,
ManufacturingDate DATE,
CompletionDate DATE,
ManufacturingStatus VARCHAR(30),
QuantityProduced INT,
ManufacturingNotes VARCHAR(255),
OrderID INT,
ProductID VARCHAR(10),
EmployeeID VARCHAR(10),
```

```

FOREIGN KEY (OrderID) REFERENCES `Order` (OrderID),
FOREIGN KEY (ProductID) REFERENCES Product (ProductID),
FOREIGN KEY (EmployeeID) REFERENCES Employee (EmployeeID)
);

-- 1.13 INVENTORY (depends on Product)
CREATE TABLE Inventory (
  InventoryID INT PRIMARY KEY NOT NULL,
  QuantityInStock INT,
  StockStatus VARCHAR(30),
  LastUpdated DATE,
  ProductID VARCHAR(10),
  FOREIGN KEY (ProductID) REFERENCES Product (ProductID)
);

-- 1.14 MATERIAL (depends on Supplier)
CREATE TABLE Material (
  MaterialID VARCHAR(10) PRIMARY KEY NOT NULL,
  MaterialName VARCHAR(100),
  UnitCost DECIMAL(10,2),
  SupplierID VARCHAR(10),
  FOREIGN KEY (SupplierID) REFERENCES Supplier (SupplierID)
);

-- 1.15 SLAB_INVENTORY (depends on Granite_Type)
CREATE TABLE Slab_Inventory (
  SlabInventoryID VARCHAR(10) PRIMARY KEY NOT NULL,
  SlabCode VARCHAR(50),
  Size VARCHAR(50),
  Quantity INT NOT NULL,
  UnitCost DECIMAL(10,2),
  GraniteTypeID VARCHAR(10),
  FOREIGN KEY (GraniteTypeID) REFERENCES Granite_Type (GraniteTypeID)
);

-- 1.16 SUPPLIER_MATERIAL (junction table: depends on Supplier, Material)
CREATE TABLE Supplier_Material (
  SupplierMaterialID VARCHAR(10) PRIMARY KEY NOT NULL,
  SupplierID VARCHAR(10),
  MaterialID VARCHAR(10),
  FOREIGN KEY (SupplierID) REFERENCES Supplier (SupplierID),
  FOREIGN KEY (MaterialID) REFERENCES Material (MaterialID)
);

-- 1.17 PRODUCT_MATERIAL (junction table: depends on Product, Material)
CREATE TABLE Product_Material (
  ProductMaterialID VARCHAR(10) PRIMARY KEY NOT NULL,
  ProductID VARCHAR(10),
  MaterialID VARCHAR(10),
  FOREIGN KEY (ProductID) REFERENCES Product (ProductID),
  FOREIGN KEY (MaterialID) REFERENCES Material (MaterialID)
);

```

(b) Sample data population (DML)

The corresponding INSERT INTO statements for every table (Customer, Cemetery, Supplier, Granite_Type, Employee, Product, Order, Invoice, Delivery, Order_Item, Deceased, Manufacturing, Inventory, Material, Slab_Inventory, Supplier_Material and Product_Material) are provided in full in SQLqueries-tables.sql, populated with the sample rows shown in section 1.5 above.

PART 2: DATA WAREHOUSE DESIGN

Part 2 builds directly on the Part 1 operational database. It designs a data warehouse that reorganises the operational sales data using dimensional modelling, so that management can run fast, flexible analytical queries.

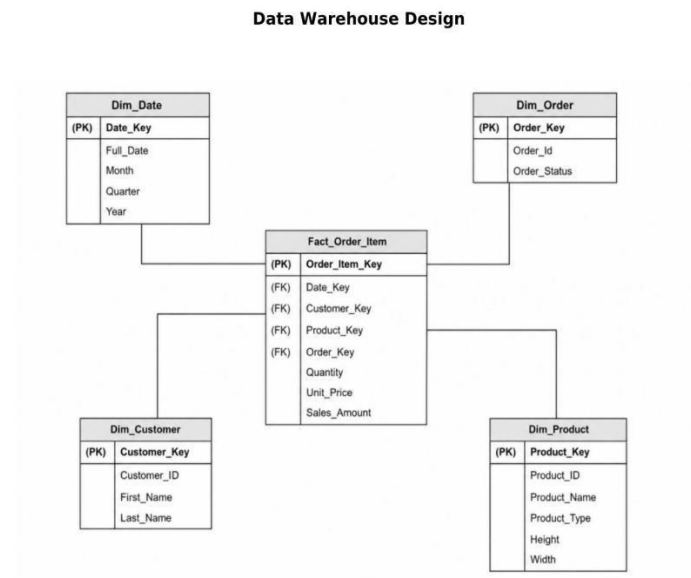


Figure 2: Star schema for the Tombstone Services data warehouse (Fact_Order_Item with four conformed dimensions).

2.1 Facts and Dimension Tables

Fact Table

Fact_Order_Item stores individual products purchased within customer orders. It contains:

- Order_Item_Key as the primary key.
- Foreign keys for Date, Customer, Product, and Order.
- Measures: Quantity, Unit_Price, and Sales_Amount.

Dim_Date

- Full_Date
- Month
- Quarter
- Year

Allows the analysis of sales over time.

Dim_Product

- Product_Key, Product_ID
- Product_Type, Product_Name
- Height and Width

Highlights the details of the product and the type of product sold.

Dim_Order

- Order_Key, Order_Id
- Order_Status

Allows the analysis of sales according to order status.

Dim_Customer

- Customer_Key, Customer_ID
- First_Name, Last_Name

Describes the customer making the purchase.

2.2 Design Choice Explained

The Star Schema was chosen because it provides a clear and practical structure for sales analysis. One central Fact_Order_Item table is connected directly to four denormalised dimension tables (Dim_Date, Dim_Customer, Dim_Product and Dim_Order), which keeps joins simple and query performance fast. The design supports business intelligence, reporting, and management decision-making across the company's sales and order-fulfilment process.

PART 3: DATA MARTS

Two business-process data marts were derived from the Part 2 data warehouse, in line with the project brief: one for the core business process (sales) and one for a secondary/management business process (order fulfilment monitoring), both built from the single Fact_Order_Item fact table and its conformed dimensions.

3.1 Data Mart 1: Sales Performance Mart (Core Process)

Purpose: gives sales and management staff a pre-aggregated view of revenue and quantity sold by product and by month. Grain: one row per product per month.

Key indicators / metrics

- Total Revenue – sum of Sales_Amount per product per month
- Total Quantity Sold – sum of Quantity per product per month
- Line Items – count of order items, indicating how many separate purchases contributed
- Average Sale Value – Total Revenue divided by Line Items

Derivation SQL (from the Part 2 warehouse)

```
CREATE VIEW Mart_Sales_Performance AS
SELECT
  d.Year, d.Month,
  p.Product_Type, p.Product_Name,
  SUM(f.Sales_Amount)           AS Total_Revenue,
  SUM(f.Quantity)             AS Total_Quantity_Sold,
  COUNT(f.Order_Item_Key)     AS Line_Items,
  SUM(f.Sales_Amount) / COUNT(f.Order_Item_Key) AS Avg_Sale_Value
FROM Fact_Order_Item f
JOIN Dim_Date d   ON f.Date_Key = d.Date_Key
JOIN Dim_Product p ON f.Product_Key = p.Product_Key
GROUP BY d.Year, d.Month, p.Product_Type, p.Product_Name;
```

Sample output (July 2026)

Product	Product Type	Total Revenue (R)	Qty Sold	Line Items	Avg Sale Value (R)
Rustenburg Grey Flat Marker	Lawn Plaque	4 500.00	1	1	4 500.00
Zimbabwe Black Classic	Headstone	12 000.00	1	1	12 000.00
African Red Double Memorial	Full Monument	3 500.00	1	1	3 500.00

Business value: management can see immediately that Headstones generate the highest revenue per sale, which supports decisions on stock priorities, marketing focus and pricing for the different monument types.

3.2 Data Mart 2: Order Fulfilment Mart (Secondary/Management Process)

Purpose: gives management visibility into how much order value sits at each stage of the fulfilment pipeline, supporting production and workflow planning. Grain: one row per order status per month.

Key indicators / metrics

- Orders Count – number of distinct orders in each status
- Value in Pipeline – total Sales_Amount tied up in each status
- Share of Pipeline (%) – each status's value as a percentage of total order value

Derivation SQL (from the Part 2 warehouse)

```
CREATE VIEW Mart_Order_Fulfilment AS
SELECT
  d.Year, d.Month,
  o.Order_Status,
  COUNT(DISTINCT o.Order_Key) AS Orders_Count,
  SUM(f.Sales_Amount) AS Value_In_Pipeline,
  SUM(f.Sales_Amount) * 100.0 /
    SUM(SUM(f.Sales_Amount)) OVER (PARTITION BY d.Year, d.Month) AS Share_Of_Pipeline_Pct
FROM Fact_Order_Item f
JOIN Dim_Date d ON f.Date_Key = d.Date_Key
JOIN Dim_Order o ON f.Order_Key = o.Order_Key
GROUP BY d.Year, d.Month, o.Order_Status;
```

Sample output (July 2026)

Order Status	Orders	Value in Pipeline (R)	Share of Pipeline
Quality Check	1	4 500.00	22.5%
In Production	1	12 000.00	60.0%
Ready	1	3 500.00	17.5%

Business value: the mart flags that 60% of order value is currently tied up 'In Production', pointing management towards checking capacity and lead times for that stage before it becomes a bottleneck, rather than waiting for a customer complaint about a late delivery.

Summary of Business Value



Together, the two data marts turn the raw operational sales records captured in Part 1 into decision-ready information. The Sales Performance Mart directly serves the company's core revenue-generating process, helping management track income by product line. The Order Fulfilment Mart serves the secondary/management process of monitoring production workflow, helping identify where order value and effort are concentrated at any point in time. Because both marts are built from the same conformed Date, Product and Order dimensions in the shared warehouse, results from the two marts can be compared side by side, for example checking whether the highest-revenue product line is also the one most often stuck in production.

REFERENCES

- Durban University of Technology (n.d.) Project database schema. Faculty of Accounting & Informatics. Unpublished project material.
- Elmasri, R. and Navathe, S.B. (2016) Fundamentals of Database Systems. 7th edn. Boston: Pearson.
- Kimball, R. and Ross, M. (2013) The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling. 3rd edn. Indianapolis: Wiley.